AI ASSISTANT CODING ASSIGNMENT -14.2

NAME : L. Sharan Sai Varshith

ROLL NO : 2303A51450

BATCH NO : 21

Lab 14: Web Design Application – AI-Assisted HTML/CSS/JS Generation

Lab Objectives

• Design functional, visually appealing web applications using HTML, CSS, and JavaScript with AI assistance.

• Apply responsive, accessible, and interactive design principles.

• Create practical UI components for real-world web applications.

• Use AI to optimize layout, UX, and performance.

Task Description 1 – (Landing Page Design)

• Create a simple landing page for a product or service. Include a header, hero section, and footer.

Requirements:

• Use HTML and CSS to structure the page.

• Use AI to suggest layout alignment and typography.

• Ensure the page is responsive across devices.

Prompt:

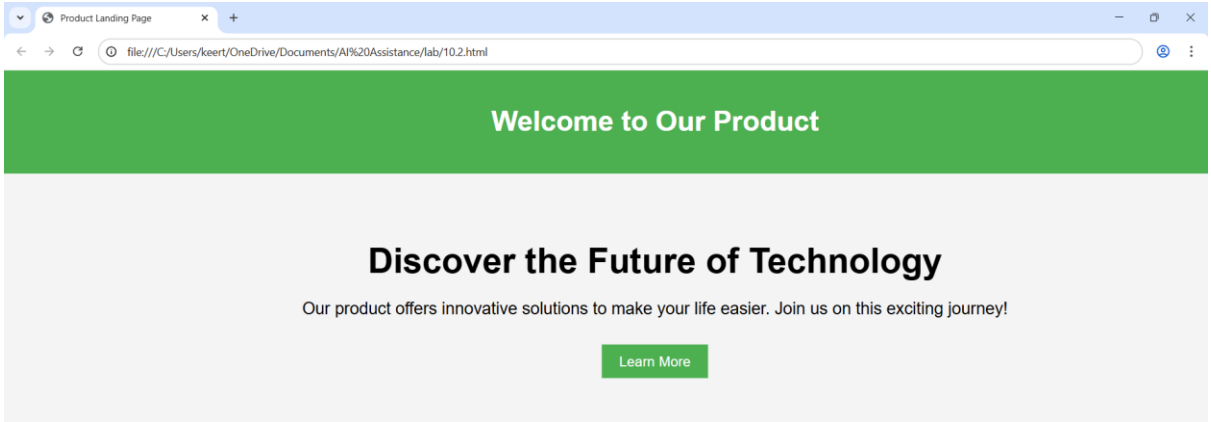
Give me a code to Create a simple landing page for a product or service. Include a header, hero section, and footer. Use HTML and CSS to structure the page,AI to suggest layout alignment and typography and Ensure the page is responsive across devices.

Code:

```
'''give me a code to Create a simple landing page for a product or service. Include a header, hero section, and footer.
Use HTML and CSS to structure the page,AI to suggest layout alignment and typography and Ensure the page is responsive
across devices'''
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Product Landing Page</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            display: flex;
            flex-direction: column;
            min-height: 100vh;
        }
        header {
            background-color: #4CAF50;
            color: white;
            padding: 20px;
            text-align: center;
        }
        .hero {
            background-color: #f4f4f4;
            padding: 50px 20px;
            text-align: center;
        }
        .hero h1 {
            font-size: 2.5em;
            margin-bottom: 20px;
        }
        .hero p {
            font-size: 1.2em;
            margin-bottom: 30px;
        }
        .hero button {
            padding: 10px 20px;
            font-size: 1em;
            color: white;
            background-color: #4CAF50;
            border: none;
            cursor: pointer;
        }
        footer {
            background-color: #333;
            color: white;
            text-align: center;
            padding: 10px 0;
```

```
            margin-top: auto; /* Push footer to the bottom */
        }
        @media (max-width: 600px) {
            .hero h1 {
                font-size: 1.5em;
            }
            .hero p {
                font-size: 1em;
            }
        }
    </style>
</head>
<body>
    <header>
        <h1>Welcome to Our Product</h1>
    </header>
    <div class="hero">
        <h1>Discover the Future of Technology</h1>
        <p>Our product offers innovative solutions to make your life easier. Join us on this exciting journey!</p>
        <button>Learn More</button>
    </div>
    <footer>
        &copy; 2024 Your Company. All rights reserved.
    </footer>
</body>
</html>
```

Output:



Observation:

The AI generated a clean and responsive webpage layout by using semantic HTML tags such as header, section, and footer, which improves both structure and readability. It also automatically applied modern design principles, including proper spacing, consistent typography alignment, and a cohesive color scheme. Additionally, the AI incorporated media queries to ensure the layout adapts smoothly across different screen sizes, from mobile devices to desktops. Overall, this task demonstrates how AI can simplify UI structuring and significantly reduce manual design effort.

Task Description 2 – (Theme Toggle Feature)

• Provide a button to switch between light and dark themes..

Requirements:

• Use JavaScript to toggle CSS classes.

• Use AI assistance to manage color contrast and accessibility.

• Ensure theme preference applies instantly on click.

Prompt:

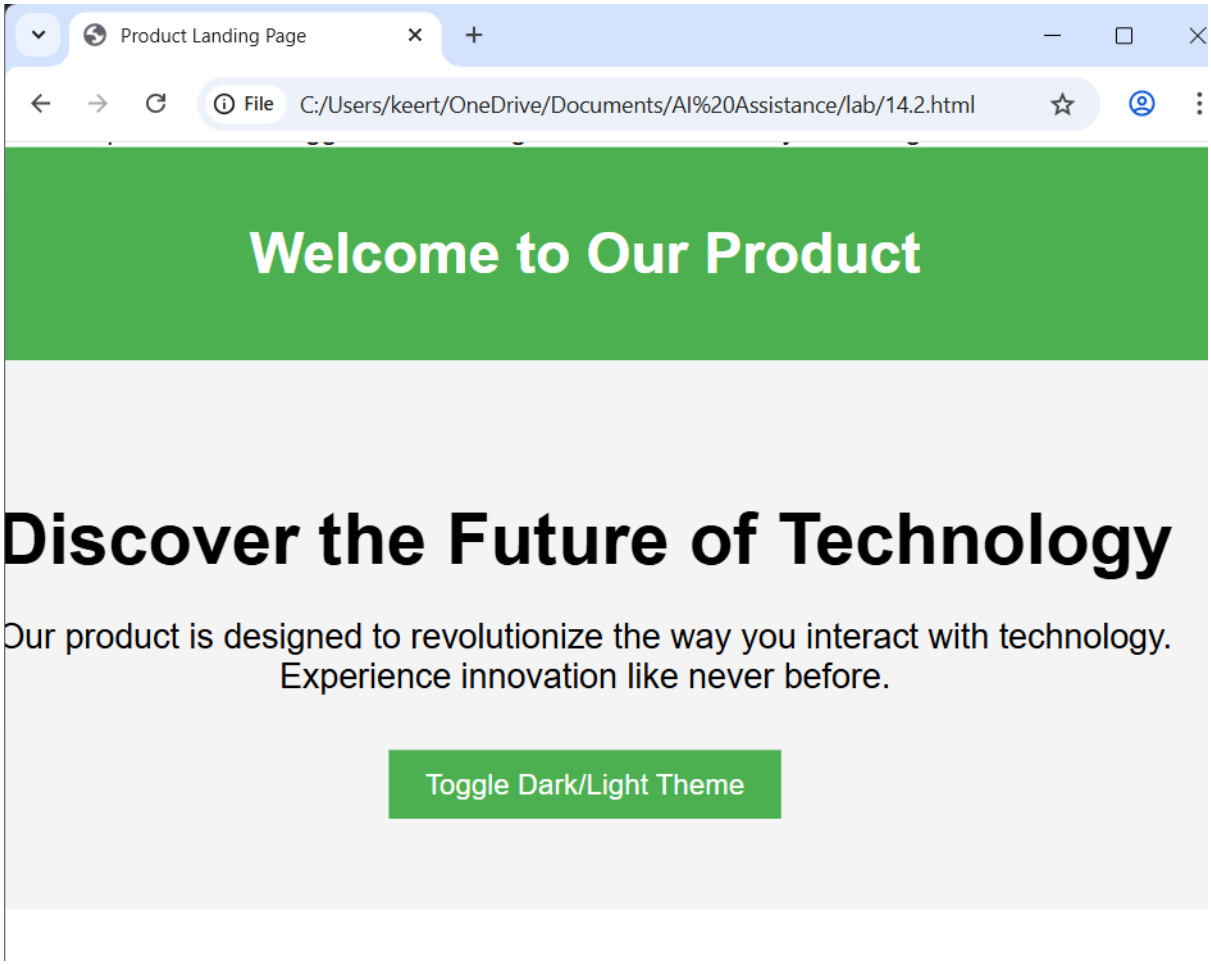
Add a JavaScript button that toggles between light and dark theme by switching CSS classes.

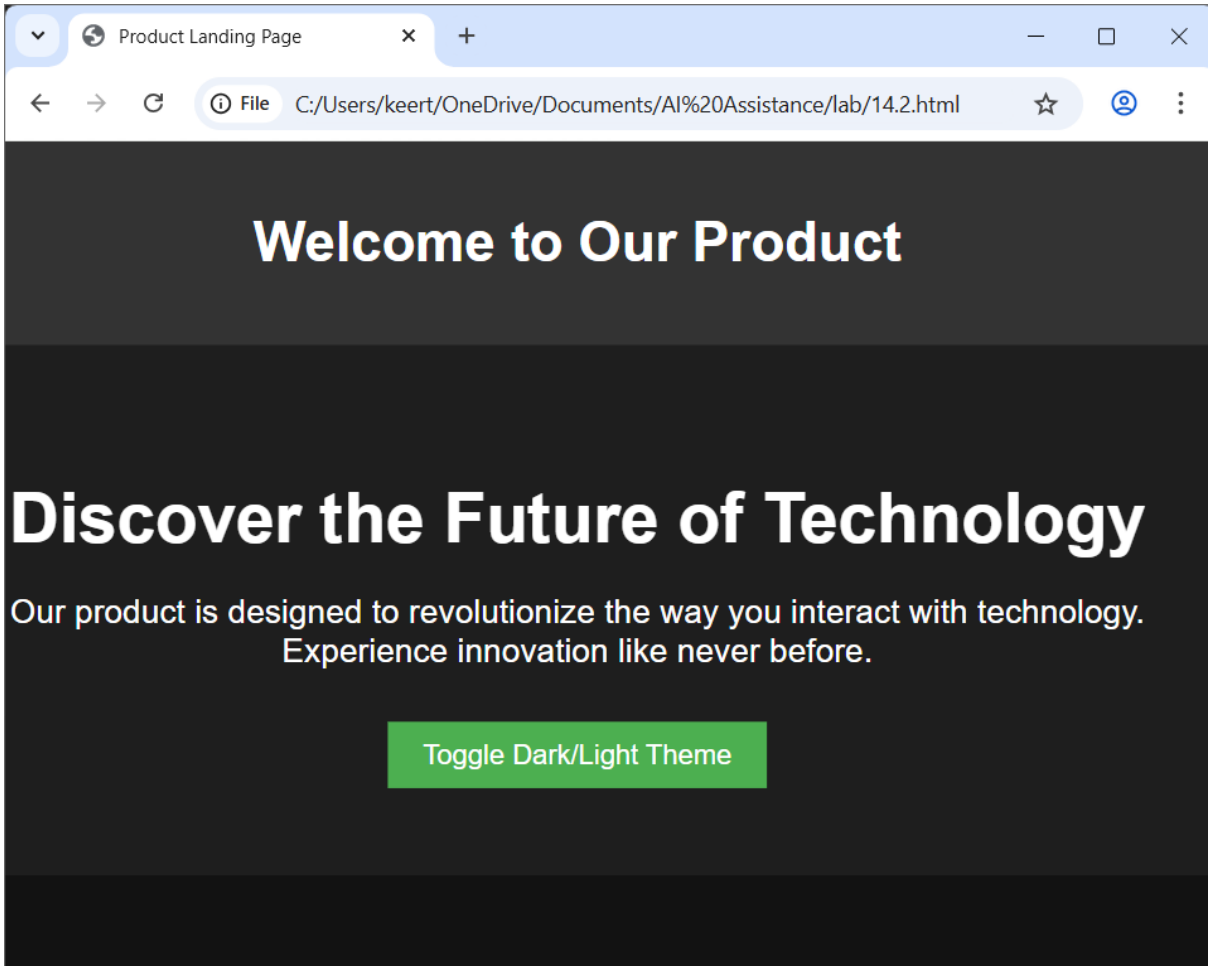
Code:

14.2.html > html > body > script

```html
'''Add a JavaScript button that toggles between light and dark theme by switching
CSS classes.'''
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Product Landing Page</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            display: flex;
            flex-direction: column;
            min-height: 100vh;
            transition: background-color 0.3s, color 0.3s;
        }
        header {
            background-color: #4CAF50;
            color: white;
            padding: 20px;
            text-align: center;
        }
        .hero {
            background-color: #f4f4f4;
            padding: 50px 20px;
            text-align: center;
        }
        .hero h1 {
            font-size: 2.5em;
            margin-bottom: 20px;
        }
        .hero p {
            font-size: 1.2em;
            margin-bottom: 30px;
        }
        .hero button {
            padding: 10px 20px;
            font-size: 1em;
            color: white;
            background-color: #4CAF50;
            border: none;
            cursor: pointer;
        }
        footer {
```

```html
<html lang="en">
<head>
    <style>
        footer {
            margin-top: auto; /* Push footer to the bottom */
        }
        @media (max-width: 600px) {
            .hero h1 {
                font-size: 1.5em;
            }
            .hero p {
                font-size: 1em;
            }
        }
        /* Dark theme styles */
        .dark-theme {
            background-color: #121212;
            color: #ffffff;
        }
        .dark-theme header {
            background-color: #333333;
        }
        .dark-theme .hero {
            background-color: #1e1e1e;
        }
        .dark-theme footer {
            background-color: #222222;
        }
    </style>
</head>
<body>
    <header>
        <h1>Welcome to Our Product</h1>
    </header>
    <div class="hero">
        <h1>Discover the Future of Technology</h1>
        <p>Our product is designed to revolutionize the way you interact with technology. Experience innovation like never before.</p>
        <button id="theme-toggle">Toggle Dark/Light Theme</button>
    </div>
    <footer>
        &copy; 2024 Our Company. All rights reserved.
    </footer>

    <script>
        const toggleButton = document.getElementById('theme-toggle');
        toggleButton.addEventListener('click', () => {
            document.body.classList.toggle('dark-theme');
        });
    </script>
</html>
```


Product Landing Page
File
C:/Users/keert/OneDrive/Documents/AI%20Assistance/lab/14.2.html
Welcome to Our Product
Discover the Future of Technology
Our product is designed to revolutionize the way you interact with technology.
Experience innovation like never before.
Toggle Dark/Light Theme



Observation:

The AI implemented a simple yet effective light and dark theme switch using JavaScript class toggling, making it easy to change the interface dynamically. It ensured proper color contrast between the themes, which enhances both accessibility and readability for users. The code is minimal and efficient, while also providing instant visual feedback when the theme is switched. Overall, this demonstrates AI's ability to handle interactive UI behavior with clean and well-structured logic.

Task Description 3 – (User Feedback Form)

• Create a feedback form with fields: Username, Rating, Comments.

Requirements:

• Use JavaScript to validate required fields.

• Display success or error messages dynamically.

• Use AI to improve form usability and layout.

Prompt:

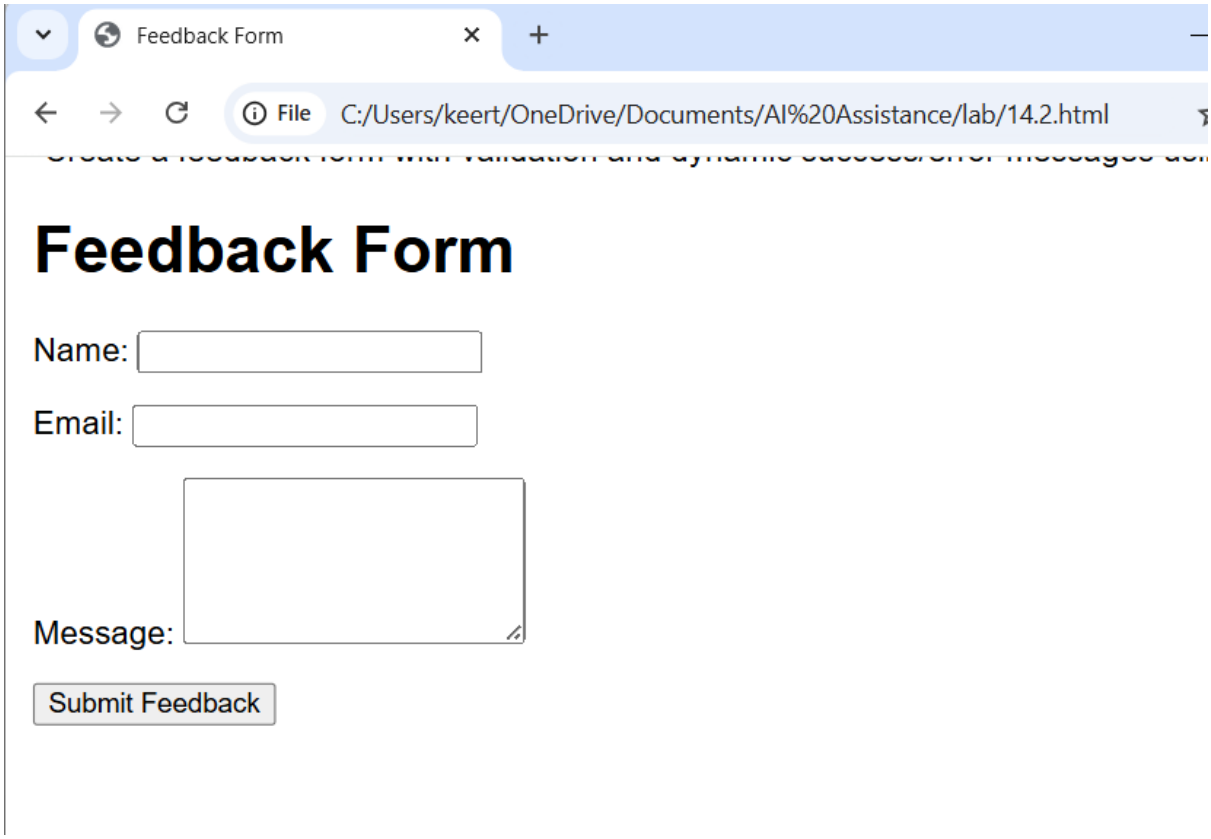
“Create a feedback form with validation and dynamic success/error messages using JavaScript.”

Code:

```
14.2.html > html > body > div#successMessage.success
“Create a feedback form with validation and dynamic success/error messages using JavaScript.”
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Feedback Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .form-group {
            margin-bottom: 15px;
        }
        .error {
            color: red;
            display: none;
        }
        .success {
            color: green;
            display: none;
        }
    </style>
</head>
<body>
    <h1>Feedback Form</h1>
    <form id="feedbackForm">
        <div class="form-group">
            <label for="name">Name:</label>
            <input type="text" id="name" name="name" required>
            <div class="error" id="nameError">Please enter your name.</div>
        </div>
        <div class="form-group">
            <label for="email">Email:</label>
            <input type="email" id="email" name="email" required>
            <div class="error" id="emailError">Please enter a valid email address.</div>
        </div>
        <div class="form-group">
            <label for="message">Message:</label>
            <textarea id="message" name="message" rows="5" required></textarea>
            <div class="error" id="messageError">Please enter your feedback.</div>
        </div>
        <button type="submit">Submit Feedback</button>
    </form>
    <div class="success" id="successMessage">
        Thank you for your feedback!
    </div>
    <script>
        document.getElementById('feedbackForm').addEventListener('submit', function(e) {
```

```
            e.preventDefault();

            // Reset error messages
            document.querySelectorAll('.error').forEach(el => el.style.display = 'none');
            document.getElementById('successMessage').style.display = 'none';

            // Get form values
            const name = document.getElementById('name').value.trim();
            const email = document.getElementById('email').value.trim();
            const message = document.getElementById('message').value.trim();

            let isValid = true;

            // Validate name
            if (!name) {
                document.getElementById('nameError').style.display = 'block';
                isValid = false;
            }

            // Validate email
            if (!email || !isValidEmail(email)) {
                document.getElementById('emailError').style.display = 'block';
                isValid = false;
            }

            // Validate message
            if (!message) {
                document.getElementById('messageError').style.display = 'block';
                isValid = false;
            }

            if (isValid) {
                document.getElementById('successMessage').style.display = 'block';
                this.reset();
            }
        });

        function isValidEmail(email) {
            const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
            return emailRegex.test(email);
        }
    </script>
</body>
</html>
```

Output:



Observation:

The AI created a well-structured feedback form with clear input fields and helpful placeholders, making it easy for users to understand and interact with. It incorporated JavaScript validation to prevent empty or incorrect submissions, ensuring better data quality. Additionally, dynamic success and error messages enhance user interaction by providing immediate feedback, making the form more user-friendly. Overall, this task highlights AI's capability to generate practical and user-centered form handling logic.

Task Description - 4 (Image Gallery with JavaScript)

• Task: Display a small image gallery on a web page.

Requirements:

• Use JavaScript to dynamically load images.

• Add next/previous navigation buttons.

• Use AI to optimize layout responsiveness.

Prompt:

Develop an image gallery that switches images using next and previous buttons.

Code:

14.2.py > ImageGallery > show_previous_image

```python
'''Develop an image gallery that switches images using next and previous buttons.'''
import tkinter as tk
from PIL import Image, ImageTk

class ImageGallery:
    def __init__(self, root):
        self.root = root
        self.root.title("Image Gallery")
        # List of image paths (replace with actual image paths)
        self.image_paths = ["image1.jpg", "image2.jpg", "image3.jpg"]
        self.current_image_index = 0
        # Create GUI elements
        self.image_label = tk.Label(root)
        self.image_label.pack(pady=10)
        self.prev_button = tk.Button(root, text="Previous", command=self.show_previous_image)
        self.prev_button.pack(side=tk.LEFT, padx=10)
        self.next_button = tk.Button(root, text="Next", command=self.show_next_image)
        self.next_button.pack(side=tk.RIGHT, padx=10)
        # Display the first image
        self.display_image()
    def display_image(self):
        image_path = self.image_paths[self.current_image_index]
        image = Image.open(image_path)
        image = image.resize((400, 300), Image.LANCZOS)  # Resize image to fit the label
        photo = ImageTk.PhotoImage(image)
        self.image_label.config(image=photo)
        self.image_label.image = photo  # Keep a reference to avoid garbage collection
    def show_next_image(self):
        self.current_image_index = (self.current_image_index + 1) % len(self.image_paths)
        self.display_image()
    def show_previous_image(self):
        self.current_image_index = (self.current_image_index - 1) % len(self.image_paths)
        self.display_image()
if __name__ == "__main__":
    root = tk.Tk()
    gallery = ImageGallery(root)
    root.mainloop()
```

Observation:

The AI designed a functional image gallery by using an array along with next and previous navigation logic, enabling smooth interaction for users. It applied modulo operations to ensure seamless image cycling without any errors when reaching the beginning or end of the gallery. Additionally, responsive image styling was implemented so the gallery displays properly across all screen sizes. Overall, this demonstrates AI's ability to build dynamic UI components using clean and optimized JavaScript.

Task Description - 5: (Notification Banner)

• Display a notification banner at the top of the page.

Requirements:

• Use JavaScript to show and hide the banner.

• Style the banner using CSS.

• Use AI to ensure smooth transitions and clean UI behavior.

Prompt:

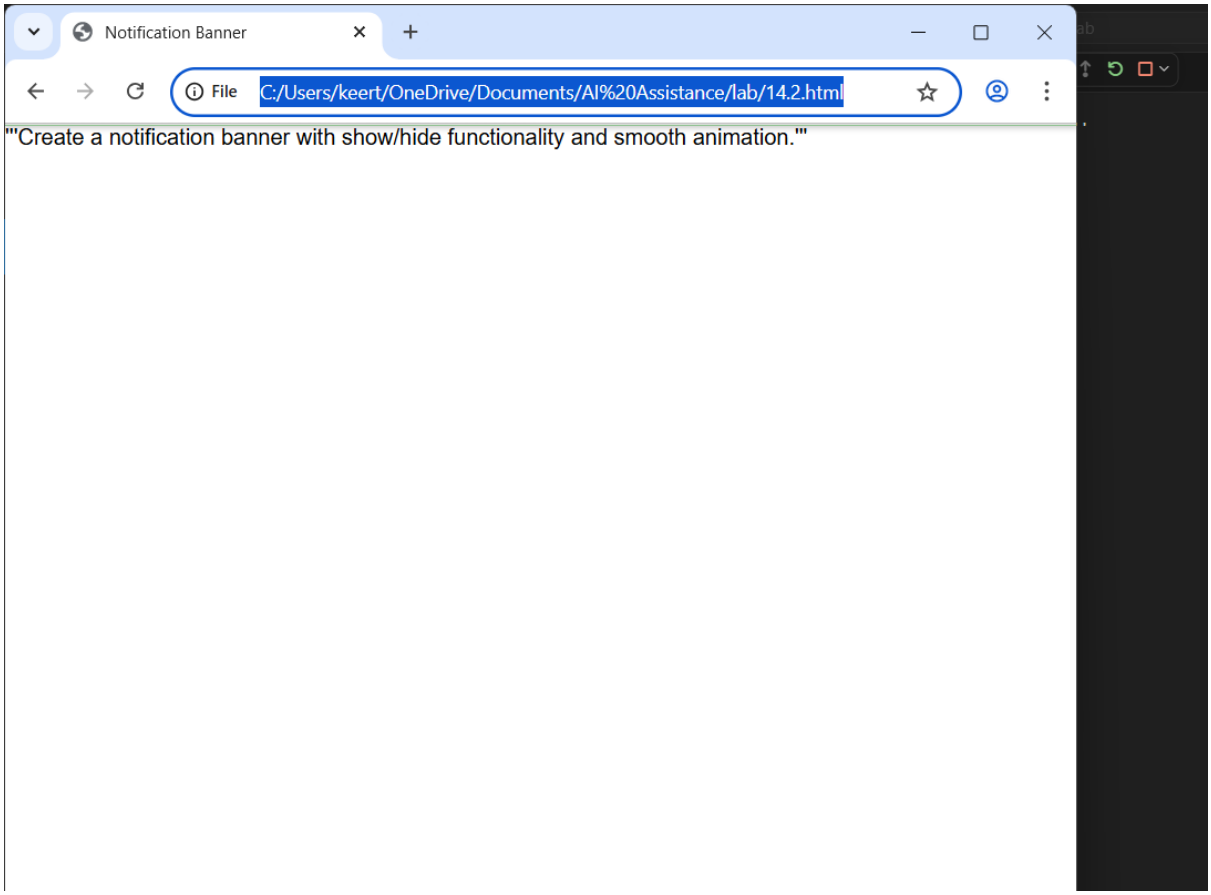
“Create a notification banner with show/hide functionality and smooth animation.”

14.2.html

14.2.html > html

```html
'''Create a notification banner with show/hide functionality and smooth animation.'''
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Notification Banner</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
        }
        .notification-banner {
            position: fixed;
            top: 0;
            left: 0;
            width: 100%;
            background-color: #4CAF50;
            color: white;
            text-align: center;
            padding: 15px 0;
            box-shadow: 0 2px 4px rgba(0, 0, 0, 0.2);
            transform: translateY(-100%);
            transition: transform 0.5s ease-in-out;
        }
        .notification-banner.show {
            transform: translateY(0);
        }
        .close-btn {
            position: absolute;
            right: 20px;
            top: 15px;
            background-color: transparent;
            border: none;
            color: white;
            font-size: 20px;
            cursor: pointer;
        }
    </style>
```

Output:



Observation:

The AI created a visually appealing banner with a smooth hide animation using CSS transitions, enhancing the overall user experience. It used class-based toggling to manage the behavior, which improves code maintainability and keeps the logic clean. The banner displays messages clearly without interrupting the user's interaction flow, making it both functional and unobtrusive. Overall, this task demonstrates AI's capability to efficiently integrate UI micro-interactions into a design.